Foundation of Deep Learning Project
Academic Year 2025-2026

Multi-Class Image Classification

An Incremental Deep Learning Approach to Flower Classification

Authors

Anna Rebecca Sala - MAT. 894569
Giorgia Brioschi - MAT. 894407
Riccardo Furgeri - MAT. 895726

Introduction

The Problem We're Solving

The Challenge

Task
Multi-class image classification (5 flower categories)

Dimensionality problem
Digital images are 3D volumes (W×H×C). A standard input generates a massive feature space: 256×256×3 = 196,608 raw values per image!

Traditional MLP Limits
Full connectivity causes parameter explosion, computational inefficiency and overfitting.

The Approach

Solution
CNN to exploit local connectivity and receptive fields through convolutional filters to drastically reduce parameter counts.

Methodology
An incremental 5-step path: from a minimal baseline model to advanced optimization and Transfer Learning.

Performance evaluation
Models evaluated via loss function and accuracy metrics, highlighting peak performance at the specific epoch with the lowest validation loss.

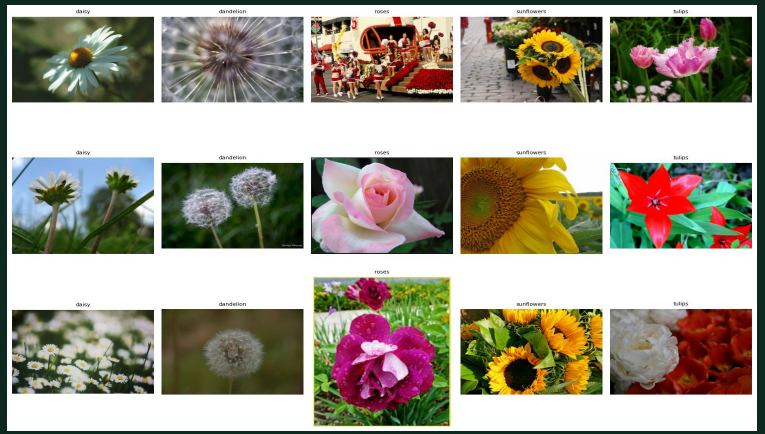
Data Exploration

Dataset & Class Balance

Dataset Overview

TensorFlow "Flowers" Dataset

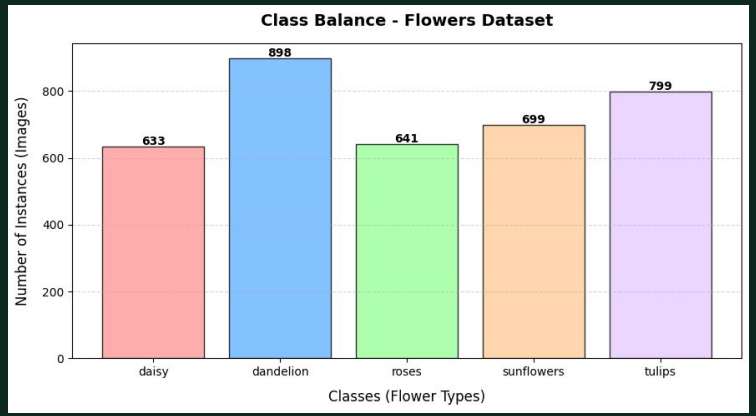
Consists of 3,670 images total (218 MB) in standard JPEG format. This set serves as our core foundation for multi-class model training.



Class Balance Analysis

Partially Uneven Distribution

5 classes ranging from 898 (dandelions) to 633 (daisies). The slight imbalance is manageable but critical for evaluating our final validation metrics



Data Exploration

Image Variability & Preprocessing

Dimensional Variability

Geometric analysis of all the images

Width
143 to 1024 px (Avg: 365.06 px)

Height
159 to 442 px (Avg: 271.79 px)

Aspect Ratio
0.60 to 5.48 (Avg: 1.33)

Preprocessing Strategy

Standardizing input data for the model

Standardization Target

224 × 224 × 3 (RGB)

Resizing Operations
Deep learning networks require consistent input dimensions to operate, so all images are geometrically scaled to a fixed resolution.

Normalization
Pixel intensities are normalized to standardize the input range and eliminate magnitude discrepancies between features, and guarantee faster model convergence.

Model 1

The Baseline

Model Architecture

The Technical Setup

- **Input Layer:** Fixed volume of (224, 224, 3)
- **Feature Extraction:** 1 single Conv2D layer (32 filters, 3×3, "same") + ReLU
- **Pooling:** GlobalMaxPooling2D to flatten volume
- **Output Layer:** Dense layer with Softmax

Core Idea & Setup

Benchmark & Training Strategy

- **Starting Benchmark:** Bare-minimum network to establish baseline performance.
- **The Goal:** Beat pure random guessing (20% accuracy threshold) with basic features.
- **Optimization:** categorical_crossentropy | RMSprop (LR: 0.001)
- **Training Run:** Fixed budget of 40 epochs.

Performance Evaluation 1

The Baseline

Performance Overview

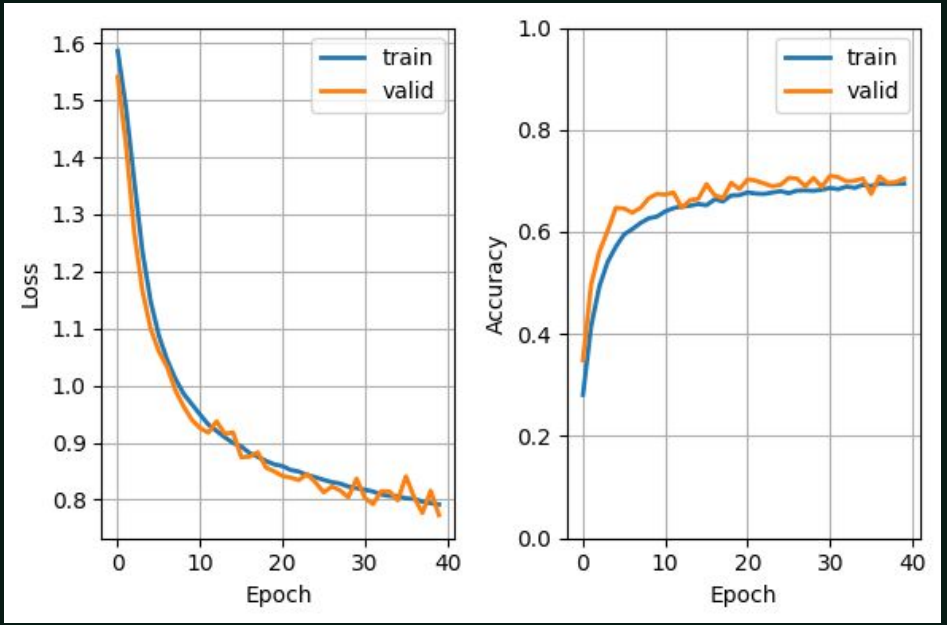
The baseline model displays low overall accuracy and highly unstable optimization steps.

Key Strengths & Weaknesses

Strength: Fast execution & extremely low overhead per epoch.
Weakness: Severe underfitting; lacks structural capacity for complex visual patterns.

Peak Performance Metrics (Epoch 40)

Training: 69.45% Accuracy | 0.7916 Loss
Validation: 70.44% Accuracy | 0.7729 Loss



Model 2

Deeper CNN

Model Architecture

The Technical Setup

- **Input Layer:** Fixed volume of (224, 224, 3)
- **Conv Cascade:** 3 progressive Conv2D layers + ReLU (32 → 64 → 128 filters)
- **Pooling Hierarchy:** 2 intermediate MaxPooling2D (3×3, stride 3) + final GlobalMaxPooling2D
- **Output Layer:** Dense layer with Softmax for final category prediction
- **Optimization:** Identical to Model 1 (RMSprop, LR 0.001, 40 epochs)

Core Idea & Evolution

Paradigm Shift & Hierarchy

- **Resolving Underfitting:** Transitioning from a single layer to a multi-layered hierarchical network to dramatically expand representation capacity.
- **Feature Hierarchy:** Early layers detect raw boundaries, while deeper layers merge those boundaries into abstract geometric shapes.
- **Spatial Reduction:** The introduction of stride-3 Max Pooling rapidly downsamples data dimensions, forcing subsequent layers to view features globally.

Performance Evaluation 2

Deeper CNN

Performance Overview

Expanding depth increases capacity but triggers severe divergence. Training approaches near-perfection while validation loss degrades.

Key Strengths & Weaknesses

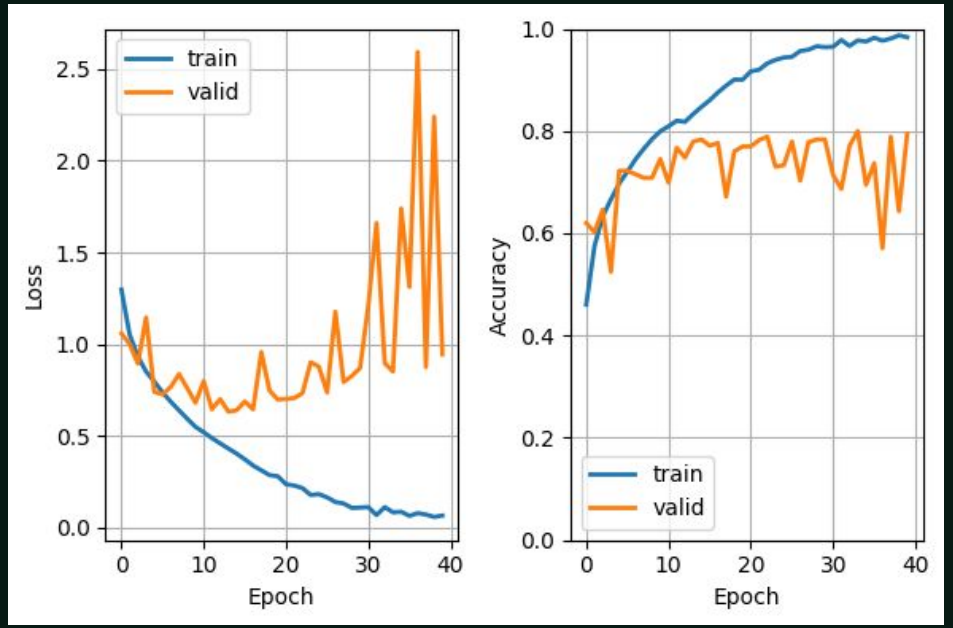
Strength: High learning capacity with deeper geometric feature extraction.

Weakness: severe overfitting; memorizes training noise without generalizing.

Peak Performance Metrics (Epoch 14)

Training: 83.31% Accuracy | 0.4302 Loss

Validation: 77.93% Accuracy | 0.6316 Loss



Model 3

Regularized CNN

The Technical Setup

Architecture & Safeguards

- **Base Structure:** 3-layer deep CNN (32 → 64 → 128 filters)
- **Regularization:** `l2(0.001)` applied to all Conv & Dense layers
- **Stability:** `BatchNormalization()` before every `ReLU`
- **Dense Expansion:** 64-unit hidden layer before classification
- **Early Stopping:** `val_loss` watchdog with `patience=5`
- **Training Run:** Expanded to 100 epochs with safety callback

The Core Idea & Evolution

Paradigm Shift & Overfitting Controls

- **Suppressing Overfitting:** Model 3 injects structural and mathematical checkpoints to stop the network from memorizing pixel noise.
- **L2 Regularization**: Penalizes large weights to prevent neurons from memorizing noise.
- **Batch Normalization:** Normalizes hidden layer activations to accelerate and stabilize training.
- **Early Stopping:** Automatically halts training when validation loss stops improving, preserving the best generalized weights.

Performance Evaluation 3

Regularized CNN

Performance Overview

The mathematical safeguards successfully stabilize the optimization landscape. Curves converge smoothly, and Early Stopping halts at Epoch 16.

Key Strengths & Weaknesses

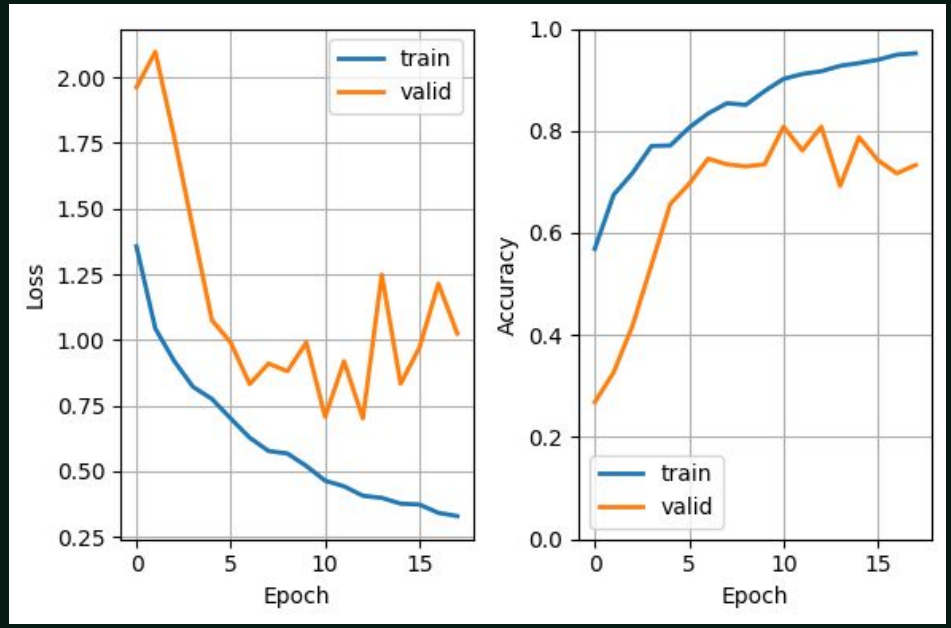
Strength: Strong validation stability, reduced overfitting variance, and fast convergence via Batch Normalization.

Weakness: Validation accuracy is bounded by the limited scale of the raw dataset (~3,600 images).

Peak Performance Metrics (Epoch 13)

Training: 91.66% Accuracy | 0.4070 Loss

Validation: 80.79% Accuracy | 0.7018 Loss



Model 4

Augmented CNN

The Technical Setup

Data & Preprocessing Pipeline

- **Expanded Loading:** Dataset loaded at (256, 256)
- **Dynamic Input:** Keras shape set to (None, None, 3)
- **Augmentation:** RandomCrop → Flip → Contrast
- **Architecture:** Regularized 3-layer CNN (matching Model 3)
- **Optimization:** Max 100 epochs with EarlyStopping(patience=5)

The Core Idea & Evolution

Data Gym & Invariance Learning

- **Expanding Data Variety:** Overcoming the ~3,600 image limit by altering data loading directly.
- **The "Data Gym":** Randomly warping, cropping, and altering illumination on-the-fly each epoch.
- **Invariance Learning:** Forces the model to learn robust spatial & lighting patterns instead of memorizing.

Performance Evaluation 4

Augmented CNN

Performance Overview

The model exhibits slower convergence because targets mutate every epoch, but features are highly robust.

Key Strengths & Weaknesses

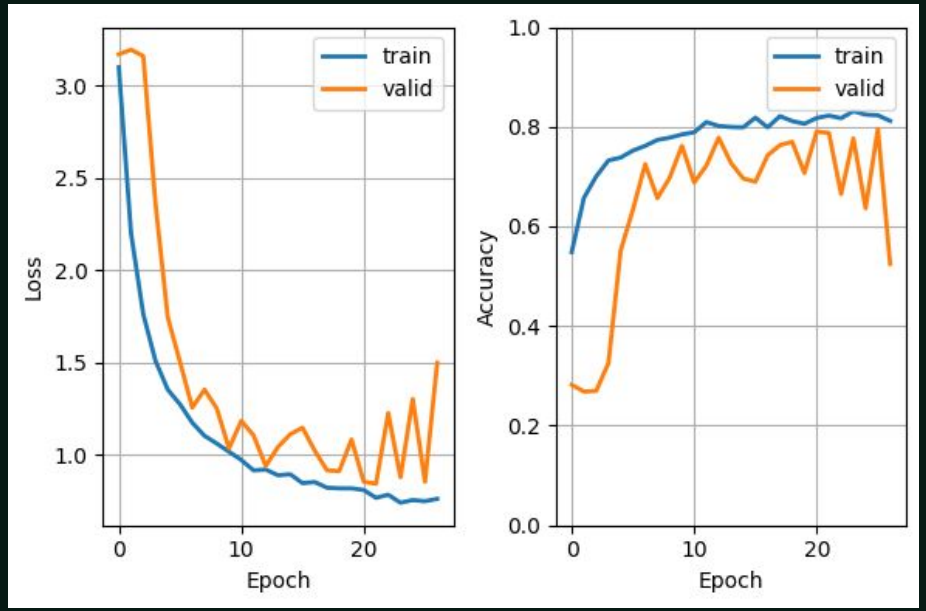
Strength: High immunity to overfitting and superior generalization across lighting and spatial variations.

Weakness: Higher computational overhead per epoch and a bounded accuracy ceiling when training a custom CNN from scratch.

Peak Performance Metrics (Epoch 11)

Training: 82.22% Accuracy | 0.7683 Loss

Validation: 78.75% Accuracy | 0.8446 Loss



Model 5

Transfer Learning

The Technical Setup

Architecture & Training Details

Architecture:

- **Model:** MobileNetV2 pre-trained on the ImageNet problem
- **Weight Freezing:** weights inherited from the pre-training (base_net.trainable=False)
- **Pre-processing:** original pre-processing procedure inherited via preprocess_input
- **Pooling:** Global Average Pooling (pooling='avg') to aggregate the features
- **Model Specialization:** final mapping to the 5 output classes via Softmax
- **Early stopping:** Quick run of 10 epochs with EarlyStopping to prevent overfitting

The Core Idea & Evolution

Transfer Learning Strategy

- **Knowledge Transfer:** Exploiting a network already trained on a different task to overcome our limited dataset.
- **Feature Preservation:** Keeping the inherited weights to preserve features learned from millions of images.
- **Training Efficiency:** Fine-tuning only the additional specialized layers to reach high performance in very few epochs.

Performance Evaluation 5

Transfer Learning

Performance Overview

Transfer Learning achieves high performance with exceptionally rapid convergence.

Key Strengths & Weaknesses

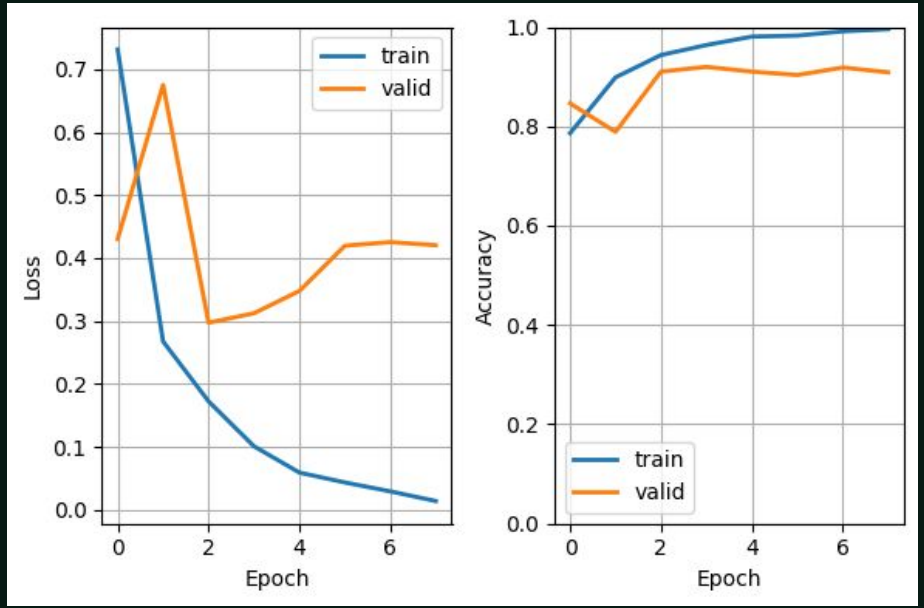
Strength: Exceptional feature extraction efficiency and minimal training time by exploiting inherited weights.

Weakness: A slight upward trend in validation loss suggests that specialized layers may begin to memorize noise if not further constrained by regularization.

Peak Performance Metrics (Epoch 3)

Training: 94.35% Accuracy | 0.1723 Loss

Validation: 91.01% Accuracy | 0.2972 Loss



Overall Consideration

Model Comparison & Key Takeaways

Model	Core Strategy	Performance Trend
1. Baseline	Single Conv2D Layer	Severe Underfitting
2. Deeper CNN	Increased Architecture Depth	Severe Overfitting
3. Regularized	L2, BatchNorm, EarlyStopping	Balanced & Optimized
4. Augmented	Random Transformations	Superior Generalization
5. Transfer Learning	**Pre-trained MobileNetV2**	**Highest Accuracy & Efficiency**

Conclusions

Training from Scratch
Requires strict regularization and data augmentation to mitigate the limitations of small datasets (~3,600 images).

Transfer Learning
Exploiting pre-trained weights offers the most efficient approach to high performance classification when the dataset is limited.

Future Developments

Alternative & Failed Experiments

Failed Experiments

Unstable Learning Rates
Initial tests without stabilizers caused unstable loss fluctuations.

Non-Stop Training
Training deep layers without Early Stopping led to a total loss of generalization.

Training without normalization
Produced similar peak accuracy but resulted in a much noisier and less stable learning process.

Future Steps

Data Growth and Balance
Train the model on an expanded and balanced dataset with a wider variety of botanical classes to improve generalization.

App Deployment
Integrate the best architecture into a mobile application for real-time flower recognition.